



Analyse Technique : Tests de Bout en Bout (E2E)

Ce document détaille la stratégie de tests automatisés réalisée avec **Playwright**.

Contrairement aux tests unitaires qui isolent les fonctions, les tests E2E simulent un véritable utilisateur naviguant sur **GeoMeteo** pour valider l'intégrité de l'interface et du tunnel de données.



Philosophie des Tests E2E

L'objectif principal est de garantir que les trois couches de l'application (Frontend JS, Backend Flask, et API Tierce) communiquent parfaitement. Nous avons choisi **Playwright** pour sa rapidité, sa fiabilité (auto-wait) et sa capacité à s'exécuter de manière "Headless" dans notre pipeline CI/CD.



Analyse des Scénarios de Tests

Le fichier `test_e2e_cold_start.py` couvre les trois piliers critiques de l'expérience utilisateur.

1. Validation du Flux Principal (Succès)

```
def test_recherche_ville_succes(page: Page):
    page.fill("#city-input", "Marseille")
    page.click("#search-btn")
    expect(page.locator("weather-info h3")).to_contain_text("Marseille")
```

- 🔍 **Analyse Technique** : On simule la saisie et le clic. On utilise `expect` avec un timeout intégré pour attendre que le DOM se mette à jour après l'appel API.
- ✅ **Justification** : Ce test garantit que la "promesse" principale du site est tenue : afficher la météo pour une ville donnée.

2. Expérience Utilisateur & Autocomplétion

```
def test_autocompletion_ux(page: Page):
    page.fill("#city-input", "Par")
    premiere_option = page.locator("#city-suggestions option").first
    premiere_option.wait_for(state="attached", timeout=10000)
```

- 🔍 **Analyse Technique** : On teste ici l'asynchronisme. On utilise `wait_for(state="attached")` avec un timeout généreux de 10s.
- ✅ **Justification (Le défi du Cold Start)** : En environnement Cloud Free Tier (Render), le serveur peut être "endormi". Ce test est conçu pour être **résilient** : il laisse au backend le temps de se réveiller et à l'algorithme Jacobson de stabiliser le timeout avant de valider la présence des suggestions.

3. Résilience et Gestion d'Erreur

```
def test_erreur_ville_introuvable(page: Page):  
    page.fill("#city-input", "VilleQuiNexistePas123")  
    page.click("#search-btn")  
    expect(page.locator("#weather-result p")).to_contain_text("Ville introuvable")
```

-  **Analyse Technique** : On force un cas d'erreur 404 via le backend pour vérifier que le message d'erreur est correctement "propagé" jusqu'à l'utilisateur.
-  **Justification** : Une bonne application ne se contente pas de fonctionner quand tout va bien ; elle doit savoir expliquer poliment à l'utilisateur quand quelque chose ne va pas.

Choix Techniques & Optimisations

Pourquoi Playwright plutôt que Selenium ?

- **Auto-Waiting** : Playwright attend que les éléments soient actionnables avant de cliquer, éliminant 80% des tests "flaky" (instables).
- **Isolation** : Chaque test tourne dans un "Browser Context" neuf, équivalent à une navigation privée, évitant les pollutions de cache entre les tests.

Gestion des Tests "Flaky" en CI/CD

Comme indiqué dans notre ci.yaml, nous utilisons l'option `--reruns 2`.

- **Raison** : Dans un pipeline automatisé, une micro-latence réseau vers OpenWeather pourrait faire échouer un test valide. Le "Retry" automatique permet de confirmer si l'échec est réel ou purement transitoire.

Impact sur la Qualité Logicielle

Grâce à cette suite de tests :

1. **Non-régression** : Toute modification du CSS ou du JS qui masquerait un élément critique est immédiatement détectée.
2. **Confiance de Déploiement** : Le job build-and-push ne se lance que si l'interface est 100% fonctionnelle.
3. **Documentation du comportement** : Les tests servent de spécification technique vivante du comportement attendu de l'interface.

Ces tests garantissent que GeoMeteo reste "Production Ready" à chaque commit.